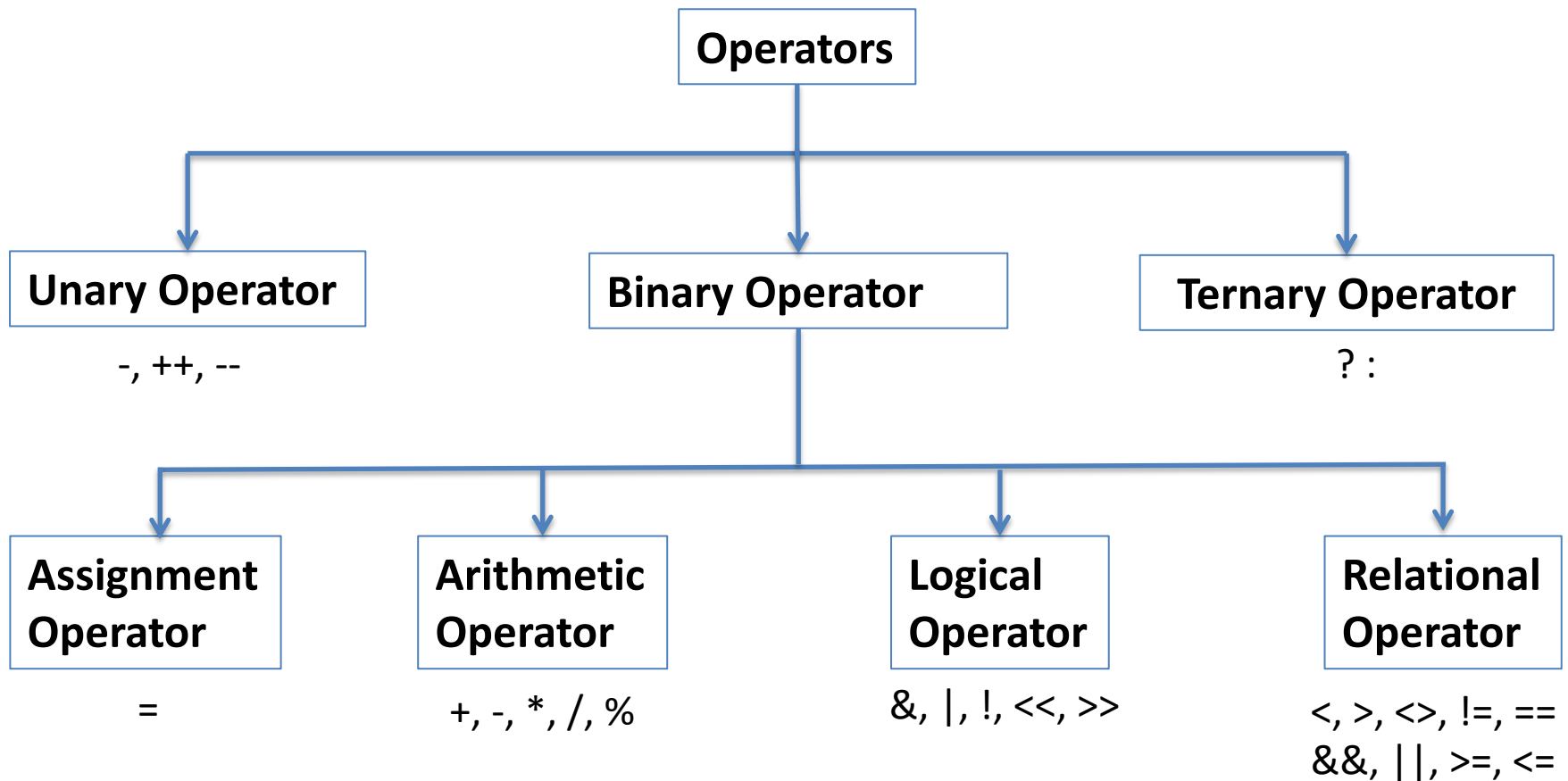


Operator

Operator

❖ **Operator:** An operator is a symbol that tells the compiler to perform certain mathematical, logical or relational manipulation.



Unary Operator

❖ **Unary Operator:** A unary operator acts upon a single operand to produce a new value.

Key unary operators are $-$ (negation), $++$ (increment), $--$ (decrement)

$-$ (negation): examples -5 , -3 , $-a$, etc.

$++$ (increment): $++a$ or $a++$ is equivalent to $a = a + 1$, i.e., the value of a will be incremented by 1. However, there is a difference between $++a$ and $a++$ when associated with other operators. For example,

$++a * b$: $++a$ will be implemented before $*$ operation.

$a++ * b$: $*$ operation will be implemented before $++a$ operation.

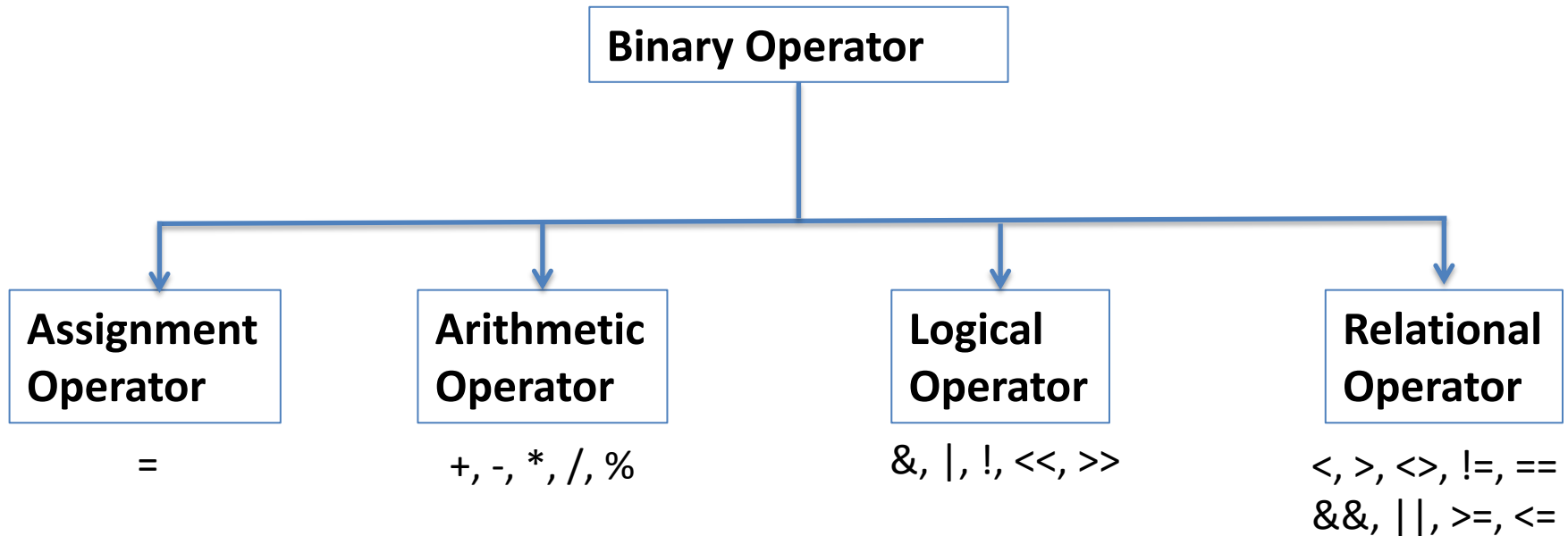
$--$ (decrement): $--a$ or $a--$ is equivalent to $a = a - 1$, i.e., the value of a will be decremented by 1. However, there is a difference between $--a$ and $a--$ when associated with other operators. For example,

$--a * b$: $--a$ will be implemented before $*$ operation.

$a-- * b$: $*$ operation will be implemented before $--a$ operation.

Binary Operator

❖ **Binary Operator:** A binary operator acts upon two operands to produce a new value.



Assignment Operator

= is called the assignment operator. This is not the equal operator. Only one variable can be placed left side of **=**, but in the right side, an expression or condition may be placed.

Expression: Some examples of expression are:

a, 2, a+2, (a+b)/2*c

Condition = exp Rop exp

exp = expression Rop = Relational Operator

Some examples of condition,

a == 2, a < b, a+2 && (a+b)/2*c

The output of condition is always either **1 (True)** or **0 (False)**.

Equation: variable = expression or condition

a=2 a=(a+b)/2*c e = a + 2 && (a+b)/2*c

Arithmetic Operator

Binary arithmetic operators are **+** (addition), **-** (minus), ***** (multiplication), **/** (division) and **%** (modulus or mod in brief).

% operator: this operator produces the remainder of division as the new value. For example, $7 \% 2 = 1$, $34 \% 10 = 4$, $7 \% 10 = 7$

Type Auto Conversion: During *Arithmetic* binary operation, if the types of two operands are different, then auto type conversion takes place in the following way:

char	->	int	int	->	float	char	->	float
------	----	-----	-----	----	-------	------	----	-------

Example:

float	S, A;	
int	B;	S = A + B;

In this example, A is float and B is int. For performing + operation, the types of two operands shall be the same. As float type cannot be converted to int, int type will be converted to float. So, the value stored in B will be converted to float and then the result will be float type.

Arithmetic Operator

Type Casting: Consider the example,

```
int    S, A;
```

```
float  B;
```

```
S = A + B;
```

- ✓ For performing addition operator, the value of A will be converted to float type and the result will be float type.
- ✓ As float type value cannot be stored in integer location, an error will be occurred with the execution of the statement, $S = A + B$;

The solution of this problem is type casting. The correct statement is given below:

```
S = (int) ( A + B);    or
```

```
S = A + (int) B;
```

Both of the above statements are correct and none of them will not generate any error.

Arithmetic Operator

Exercise:

int i, j;

short s;

double dx;

long ix;

float x;

char c;

Determine the type of following expressions: (a) $i + c$ (b) $x + c$ (c) $dx + x$
(d) $((\text{int}) dx) + ix$ (e) $i + x$ (f) $s + j$ (g) $ix + j$ (h) $s + c$ (i) $ix + c$

Solution:

(a) $i + c$ = int + char = int	(b) $x + c$ = float + char = float	(c) $dx + x$ = double + float = double	(d) $((\text{int}) dx) + ix$ = $((\text{int}) \text{double}) + \text{long}$ = int + long = long	
(e) $i + x$ = int + float = float	(f) $s + j$ = short + int = int	(g) $ix + j$ = long + int = long	(h) $s + c$ = short + char = short	(i) $ix + c$ = long + char = long

Logical Operator

- ✓ Logical operator is also known as **bitwise operator**.
- ✓ For logical operation, data must be represented into bits.

A	B	&
0	0	0
0	1	0
1	0	0
1	1	1

A	B	
0	0	0
0	1	1
1	0	1
1	1	1

A	!
0	1
1	0

Example: if $A = 13$ and $B = 18$, find (a) $A \& B$; (b) $A | B$

$A = 0000\ 0000\ 0000\ 1101$

$B = 0000\ 0000\ 0001\ 0010$

$A \& B = 0000\ 0000\ 0000\ 0000$
 $= 0$

$A = 0000\ 0000\ 0000\ 1101$

$B = 0000\ 0000\ 0001\ 0010$

$A | B = 0000\ 0000\ 0001\ 1111$
 $= 31$

<< is the left shift operation. For example, $A \ll 2$ will produce the output 0000 0000 0011 0100. **Value is multiplied by $2^2 = 4$.**

>> is the right shift operation. For example, $A \gg 2$ will produce the output 0000 0000 0000 0011. **Value is divided by $2^2 = 4$ (integer division).**

Relational Operator

❑ Relational operator always acts like a question. For example,

$a < b$ whether a is less than b or not?

Answer is always either **1 (True)** or **0 (False)**.

❑ Relational operator always makes a condition.

Exercise:

```
int i = 8, j = 5;
```

```
float x = 0.005, y = -0.01;
```

Determine the value of following expressions: (a) ! (i <= j)

(b) (x > y) && (i > 0) || (j < 5)

Solution:

a) $! (i <= j)$
= $! (8 <= 5)$
= $! (0)$
= 1

(b) $(x > y) \&\& (i > 0) || (j < 5)$
= $(0.005 > -0.01) \&\& (8 > 0) || (5 < 5)$
= $1 \&\& 1 || 0$
= $1 || 0$
= 1

Ternary Operator

❑ Example of ternary operation is,

$x = a < b ? a : b;$



```
if (a < b)
    x = a;
else
    x = b;
```

Ternary operator is also known as **conditional operator**.

Exercise: What is the value assigned to x if b is 7?

$x = b > 8 ? b << 3 : b > 4 ? b >> 1 : b;$

Solution:

(b) $x = b > 8 ? b << 3 : b > 4 ? b >> 1 : b;$

$= b > 4 ? b >> 1 : b$

[since, $b > 8$ is false]

$= b >> 1$

[since $b > 4$ is true]

$= 7 / 2^1$

$= 7 / 2$

$= 3$

Expression Evaluation

- ❖ **Precedence Rule:** Which operation will be done first?
- ❖ **Associativity Rule:** Whether the operation will be executed from left to right or right to left?

Operator Category	Operators	Associativity
Unary operators	-, ++, --, !, sizeof	R -> L
Arithmetic multiplication and division	*, /, %	L -> R
Arithmetic add and subtract	+, -	L -> R
Relational Operators	<, >, <>, <=, >=, &&, , ==	L -> R
Conditional Operators	? :	R -> L
Assignment Operators	=, +=, -=, *=, /=, %=	R -> L

Expression Evaluation

Exercise:

int i = 8, j = 5;

float x = 0.005, y = -0.01;

char c = 'c', d = 'd';

Determine the value of following expressions:

(a) $(i - 3 * j) \% (c + 2 * d) / (x - y)$

(b) $(3 * i - 2 * j) \% (2 * d - c) + (j != 6)$

Solution:

$$\begin{aligned} & (a) \ (i - 3 * j) \% (c + 2 * d) / (x - y) \\ &= (8 - 3 * 5) \% (99 + 2 * 100) / (0.005 - (-0.01)) \\ &= (8 - 15) \% (99 + 200) / (0.005 + 0.01) \\ &= -7 \% 299 / 0.015 \\ &= -7 / 0.015 \\ &= -466.67 \end{aligned}$$

$$\begin{aligned} & (a) \ (3 * i - 2 * j) \% (2 * d - c) + (j != 6) \\ &= (3 * 8 - 2 * 5) \% (2 * 100 - 99) + (5 != 6) \\ &= (24 - 10) \% (200 - 99) + 1 \\ &= 14 \% 101 + 1 \\ &= 14 + 1 \\ &= 15 \end{aligned}$$

For Your Practice

Exercise: `int      i = 8,    j = 5;`
 `float x = 0.005, y = -0.01;`
 `char c = 'c', d = 'd';`

1. Determine the type and the value of following expressions:

(a) `2 * ( ( i / 5 ) + ( j - 3 ) % ( i + j - 2 )`

(b) `( x > y ) && ( i > 0 ) && ( j < 5 )`

2. What is the value of i?

(c) `i -= ( j > 0 ) ? j : 0;`

(d) `i %= j;`